

Gelişmiş Kod Gizleme ve Çeşitlendirme

CEN429 Güvenli Programlama — Hafta 9

Dr. Öğr. Üyesi Uğur CORUH · 13.11.2026

Bugünün planı (3 saat)

Saat	Bölüm	Konu
1	1–2	Gizleme neden kural? · MATE · taksonomi · kural şablonu
2	3	Kontrol akışı kuralları (K-01...K-06)
3	4–6	Veri kuralları · sanallaştırma/dinamik · çeşitlendirme · ölçme

Kısa tarihçe — kod gizleme nereden geldi?

- **1976** — Diffie & Hellman: "anlaşılmaz ama çalışır" fikri (**çaba-yoluyla** koruma)
- **1997** — Collberg vd. **taksonomi** + *güç-dayanıklılık-gizlilik-maliyet* (bu dersin beş ailesi)
- **2001** — Barak vd.: kusursuz gizleme **imkânsızdır** → "kırılamazlık" değil **maliyet**
- **2002** — Chow vd. **whitebox AES** (11. hafta) · **2010'lar Tigress**, O-LLVM (14. hafta)

Gizleme, bir **imkânsızlık teoreminin** üzerine kurulmuş **pratik bir geciktirme** disiplini.

Nereden geldik, nereye gidiyoruz?

- **4. hafta:** sembol/dize gizleme, düzleştirmeye giriş (tanışma)
- **Bu hafta (9):** aynı teknikler **ileri** + **nasıl ölçülür**
- **11. hafta:** whitebox (anahtar için)
- **14. hafta:** aynı kuralların **otomatik** hali (Tigress)

Bu üç hafta bir bütün: kural → whitebox → otomasyon

Öğrenme çıktısı

Bu hafta **ÖÇ.3** (ikili uygulama korumaları) üstünedir.

Sonunda yapabileceğiniz:

- Gizlemeyi bir **güvenlik kuralı** olarak yazmak
- Her tekniğin **maliyet** ve **sınırını** söylemek
- Bir korumayı **ölçüp** gerekçelendirmek

1. Gizleme neden bir kuraldır?

Önce bir soru

Bir mobil uygulamayı kullanıcıya **teslim** ettiniz.

- Kod artık kimin makinesinde çalışıyor?
- Sunucudaki güvenlik denetimleriniz burada geçerli mi?
- Saldırgan kaç kez deneyebilir?

MATE: uçtaki saldırgan

Man-**A**t-**T**he-**E**nd = programın kendisine sahip saldırgan.

- İkili dosya onda
- Belleği okuyup değiştirebilir
- Tersine derleyici, hata ayıklayıcı elinde
- İsteddiği kadar dener

Buna **beyaz kutu** da denir (1. hafta).

MATE saldırgan modeli — şema

MATE: saldırgan cihazın sahibi

mobil ödeme, DRM, lisanslama: kod kullanıcının elinde çalışır

Ağ saldırganı

yalnız mesajları görür
TLS onu durdurur

klasik güvenlik dersleri
bunu varsayar

MATE (uçtaki insan)

ikiliyi açar, belleği okur
debugger bağlar, yamalar
istediği kadar dener

kripto TEK BAŞINA yetmez

Gizleme burada devreye girer: kırılmazlık değil, MALİYET yükseltme.

Kara kutu $\neq$ beyaz kutu

	Kara kutu	Beyaz kutu (MATE)
Görür	Girdi/çıktı	Her şey
Ortam	Uzak sunucu	Saldırganın cihazı
Deneme	Sınırlı	Sınırsız
Savunma	Kripto, doğrulama	+ gizleme, RASP, whitebox

Kripto **kara kutu** varsayar; teslimde bu varsayım çöker.

Dürüst gerçek

Yeterli zamanı, becerisi ve motivasyonu olan saldırgan **her gizlemeyi** çözer.

Cookbook (Tarif 12.1): anti-tampering "kırılamaz" değildir.

Öyleyse amaç ne?

Amaç: kırmayı ekonomik olmaktan çıkarmak

Gizleme bir **maliyet yükseltme** aracıdır, sır saklama değil.

- Kırmayı **pahalı** yap
- Otomatikleştirmeyi **zorlaştır**
- Diğer katmanlara **zaman kazandır**

Bunları üç kurala dökelim.

Kural 1 — Maliyeti değerin üstüne çıkar

Saldırı, korunan varlıktan **pahalı** olmalı.

- 10 TL'lik sırrı 1000 TL'lik çabayla korumak: israf
- 1000 TL'lik sırrı 10 TL'lik çabayla korumak: yetersiz

Önce varlığın değerini belirle, sonra korumayı seç.

Kural 2 — Otomasyonu kır

Bir kopyada işe yarayan saldırı, **bütün** kopyalarda işe yaramasın.

- Bunun adı **çeşitlendirme** (bu haftanın 5. bölümü)
- xiv. haftada araçla (Tigress `--Seed`)

Bir kırık, her yeri açmamalı.

Kural 3 — Diğer katmanlara zaman kazandır

Gizleme tek başına savunma **değil**; bir **katman**.

- RASP (6. hafta) çalışırken korur
- Bütünlük denetimi, sunucu denetimi
- Anahtar/sürüm yenilenene kadar **dayanmak** yeter

Kaynağın dili: her önlem bir "kural"

Teknik kılavuz her önlemi üç başlıkla yazar:

- **Açıklama** — neyi korur
- **Uygulama** — nasıl yapılır
- **Örnek kullanım** — nerede

Çoğu zaman bir **uyum kimliğiyle** (ör. "hassas veri düz loglanmaz").

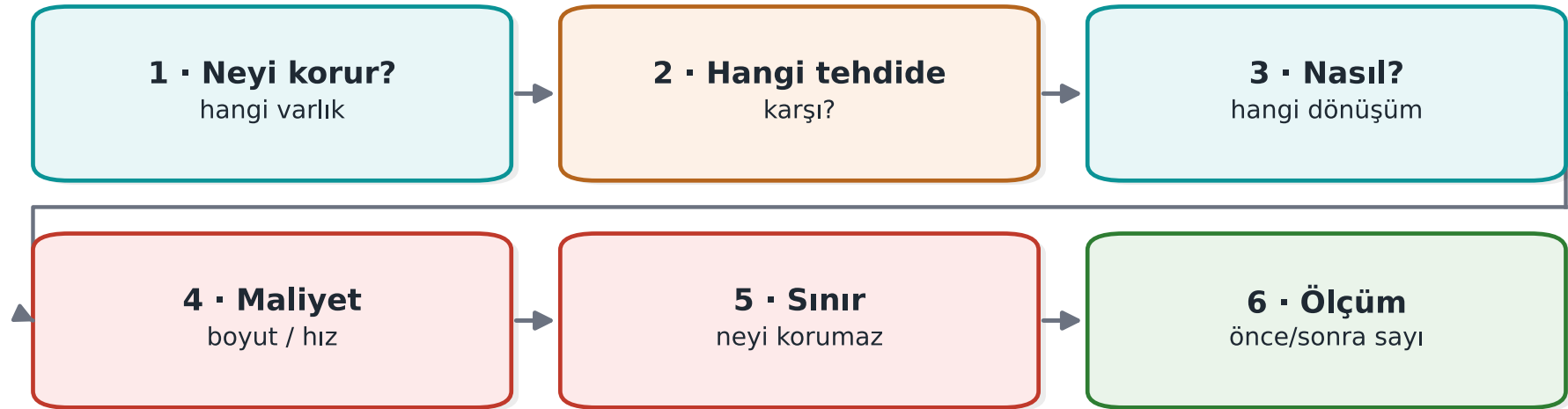
Biz de öyle yazacağız: *neyi korur · nasıl · maliyet · sınır*.

2. Taksonomi ve kural şablonu

Koruma kuralı şablonu — şema

"Koruma kuralı" şablonu

her teknik bu altı satırla yazılır — S9'a böyle gider



'Ölçüm' satırı olmayan kural, gerekçelendirilmemiş sayılır.

Gizleme neyi gizler?

Collberg'in sınıflandırması: gizlemeyi **neyi gizlediğine** göre ayırırız.

Beş aile var. Sırayla görelim.

Aile 1 — Düzen (layout)

Neyi gizler? Adları, biçimi, meta veriyi.

- Sembolleri görünmez yapmak
- Fonksiyon/dosya adlarını anlamsızlaştırmak
- Sürümde günlüğü kaldırmak

4. haftada tanıştık; bu hafta derinleştiriyoruz.

Aile 2 — Veri

Neyi gizler? Sabitleri, dizgeleri, değişkenleri.

- Dize kodlama
- Sabit dönüşümleri
- Değişken bölme/birleştirme
- Opak boolean

→ Bu haftanın 4. bölümü.

Aile 3 — Kontrol akışı

Neyi gizler? Algoritmanın yapısını.

- Kontrol akışı düzeltirme
- Opak yüklemeler
- Sahte ve ölü dallar
- Rastgele çıkış

→ Bu haftanın 3. bölümü (birazdan).

Aile 4 — Önleyici (anti-analysis)

Neyi gizler? Analiz araçlarının işini.

- Tersine derleyiciyi yanıltan yapılar
- Çalışma anında kod çözme

→ Kavram bu hafta, RASP 6. hafta.

Aile 5 — Sanallaştırma

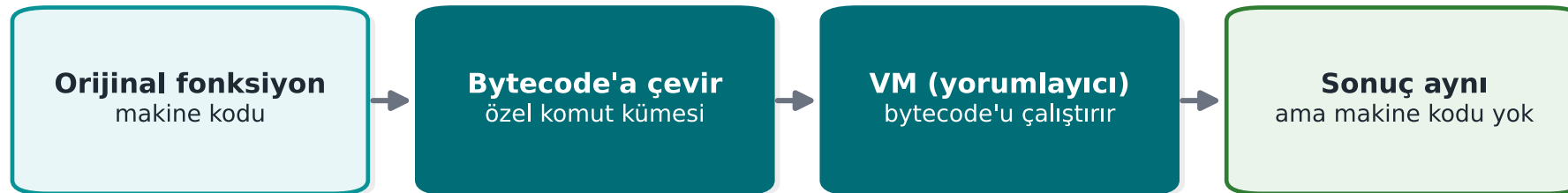
Neyi gizler? Makine kodunun kendisini.

- Fonksiyonu özel bir VM'in bayt koduna çevirmek

→ Kavram bu hafta (K-10), araçla 14. hafta (Tigress `Virtualize`).

Sanallaştırma — şema

Sanallaştırma: en güçlü, en pahalı
analizci artık sizin uydurduğunuz makineyi çözmek zorunda



Maliyet: onlarca kat yavaşlama + boyut → yalnız küçük ve kritik fonksiyonlara.

Beş aile: özet

Aile	Gizler
Düzen	Ad, biçim
Veri	Sabit, dize, değişken
Kontrol akışı	Yapı
Önleyici	Araç işini
Sanallaştırma	Makine kodu

Kural: aileler tek başına değil, **birlikte** kullanılır.

Beş aile — güç ↑ maliyet ↑

Gizlemenin beş ailesi

aşağıdan yukarı: güç ↑ · maliyet ↑



Güç birliktelikten ve çeşitlendirmeden gelir; her aile ölçülmeli. Hiçbiri anahtarı korumaz.

Sahada: hepsi bir arada

Sertifikalı bir üründe listelenen native önlemler bu haritanın **tamamını** kapsar:

sembol gizleme · ad anlamsızlaştırma · aritmetik dönüşüm · dize kodlama · opak boolean · sahte işlem · ölü dal · düzeltirme · rastgele çıkış · günlük kaldırma

Hiçbiri tek başına güçlü değil; güç **birliktelikten**.

Gizleme ne VERİR?

- Statik analize **gecikme** (`strings` , sembol, tersine derleme)
- Otomatik saldırıya **direnç** (çeşitlendirmeye)
- Diğer katmanlara **zaman**
- **Ölçülebilir** bir maliyet artışı

Gizleme ne VERMEZ?

- Bir sırrın **gizliliği** → o kriptonun işi
- Bellekten okumaya **koruma** → RASP, whitebox
- **Kalıcı** güvenlik → "kırılamaz" yoktur
- **Bedava** koruma → her gizleme maliyet getirir

En sık hata

Gizlemeyi **anahtar saklama** sanmak.

- Gömülü/gizlenmiş anahtar yine **oradadır**
- Program çalışırken onu kullanmak zorunda → bellekte açığa çıkar
- Çözme mantığı da içeride

Gizlilik kriptonun işidir, gizlemenin değil. (3. hafta)

"Koruma kuralı" şablonu

KURAL K-xx: <teknik>

Neyi korur? : hangi varlık / kod bölümü

Hangi tehdit?: statik / dinamik / otomatik / kurcalama

Nasıl? : bir cümle (kavram)

Maliyet : boyut, hız, bakım, hata ayıklama

Sınır : neyi korumaz

Ölçüm : etkinlik nasıl doğrulanır

Neden "Ölçüm" satırı şart?

Gizleme "var/yok" değildir.

- Değerlendirici (13. hafta) her katmanın saldırıyı **ne kadar geciktirdiğini** sorar
- Ölçümsüz koruma = **iddia**, kanıt değil

Projede S9'u bu şablonla yazacaksınız.

Dört ölçüt — şema

Gizlemeyi ölçen dört boyut

bir koruma kararı bu dördüyle birlikte gerekçelendirilir

GÜÇ (potency)

insan analizciyi ne kadar zorlaştırır

DAYANIKLILIK (resilience)

otomatik araçlara direnç

GİZLİLİK (stealth)

gizlemenin fark edilmezliği

MALİYET (cost)

boyut + hız cezası

Ödünleşim: dayanıklılık ↔ maliyet, güç ↔ gizlilik. Ölçmeden 'güçlendirdim' denmez.

Bölüm 2 — kısa sinama

1. MATE saldırgan kara kutudan nasıl ayrılır?
2. Gizlemenin üç kuralı neydi?
3. Gizleme bir anahtarı korur mu? Neden?

Bölüm 2 — cevaplar

1. **MATE** programa sahiptir (kod/bellek onda, sınırsız deneme); kara kutu yalnız girdi/çıktıyı görür.
2. Maliyeti değerin üstüne çıkar · otomasyonu kır (çeşitlendir) · diğer katmanlara zaman kazandır.
3. **Hayır**. Anahtar çalışırken bellekte açığa çıkar ve çözme mantığı içeridedir; gizlilik kriptonun işidir.

0. Önce temel kavramlar

Neden bu bölüm var?

Bu haftanın konusu **kod gizleme**. Ama önce birkaç temel terimi netleştirelim.

Bu terimleri bilmeden konu **havada kalır**.

Hiçbir şey bilmediğinizi varsayıyoruz — bu iyi bir başlangıç.

Kaynak kod nedir?

- Sizin yazdığınız, **insanın okuyabildiği** program metni (ör. C dosyası).
- Örnek:

```
int topla(int a, int b) { return a + b; }
```

Kaynak kod **derlenir** ve makinenin çalıştırdığı biçime döner.

Derleyici ve ikili dosya

- **Derleyici** (compiler): kaynak kodu **makine koduna** çeviren program (gcc, clang).
- **İkili dosya** (binary): derleme sonucu; bilgisayarın doğrudan çalıştırdığı dosya (`.exe` , `.so`).

```
kaynak.c --(derleyici)--> program (ikili)  
insan okur                makine çalıştırır
```

Makine kodu ve assembly

- **Makine kodu:** işlemcinin anladığı sayısal komutlar.
- **Assembly:** makine kodunun insana biraz daha okunur hali (`mov` , `cmp` , `jmp`).

İkili dosyayı açtığınızda gördüğünüz şey budur.

Tersine mühendislik nedir?

Tersine mühendislik (reverse engineering): ikili dosyaya bakıp programın **ne yaptığını** anlamaya çalışmak.

Saldırganın temel işi budur.

Tersine derleyici (decompiler)

- **Tersine derleyici** (decompiler): ikili dosyayı geri, okunabilir koda yakın bir biçime çeviren araç.
- Örnekler: Ghidra, IDA.

Amaç: kaynağa erişimi olmayan biri, ikiliye bakarak mantığı çıkarsın.

strings komutu

- **strings** : bir ikili dosyanın içindeki **okunabilir metinleri** listeleyen basit araç.
- `strings program` → `"Lisans gecersiz"`, `"http://..."` gibi metinler.

Saldırganın attığı **ilk adım** genelde budur.

Sembol ve sembol tablosu

- **Sembol:** ikili dosyadaki bir fonksiyonun ya da değişkenin **adı** (ör. `lisans_dogrula`).
- **Sembol tablosu:** bu adların listesi.

`lisans_dogrula` adı görünüyorsa, saldırgan nereye bakacağını bilir.

Hata ayıklayıcı (debugger)

- **Hata ayıklayıcı** (debugger): programı **adım adım** çalıştıran, değişkenleri gösteren araç (gdb, lldb).
- Saldırgan programı durdurup belleği okuyabilir, değerleri değiştirebilir.

Bit, bayt, onaltılık

- **Bit:** 0 veya 1.
- **Bayt:** 8 bit.
- **Onaltılık (hex):** `0x` ile yazılır; `0x2A` = 42.

Kodda `0x5A` gibi değerler göreceğiz; bunlar sadece sayılardır.

XOR nedir?

- **XOR** ($\wedge$): iki bit farklıysa 1, aynıysa 0.
- Özelliği: $a \wedge b \wedge b == a$ → **kendini geri alır.**

```
c = a ^ 0x5A; /* şifrele */  
a = c ^ 0x5A; /* geri çöz */
```

Gizlemede çok kullanılır çünkü tersine çevrilebilir.

Entropi (rastgelelik) nedir?

- **Entropi:** bir verinin ne kadar "rastgele" görüldüğü.
- Kriptografik anahtarlar **yüksek entropili** görünür (düzensiz baytlar).

Saldırgan, ikilide yüksek entropili bir blok görürse "burada anahtar olabilir" der.

Fonksiyon, dal, koşul

- **Fonksiyon:** bir işi yapan kod bloğu (`erisim_ver`).
- **Dal (branch):** `if` gibi bir yol ayrımı.
- **Koşul:** dalın hangi yöne gideceğini belirleyen ifade.

Temel blok (basic block)

- **Temel blok:** dalsız, baştan sona akan komut dizisi.
- Bir `if` gelince blok biter, iki yeni blok başlar.

Program = temel blokların birbirine bağlanması.

Kontrol akışı grafiği (CFG)

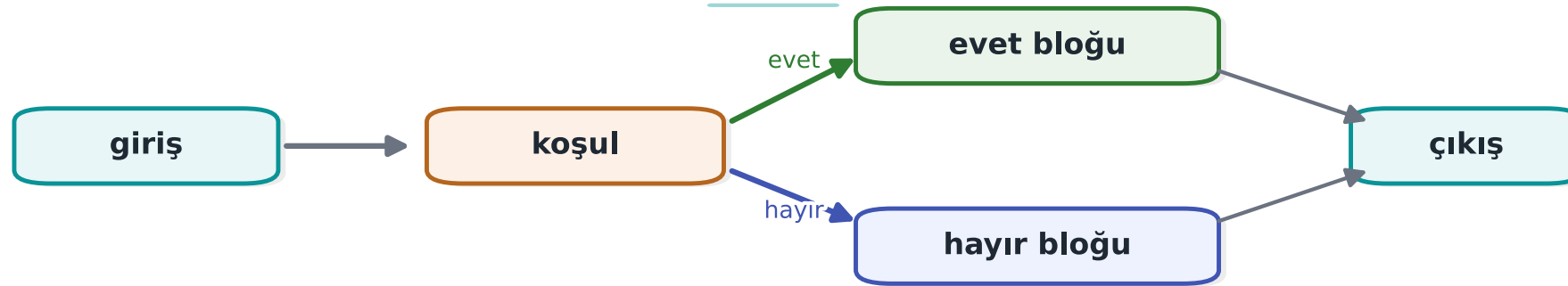
- **CFG** (Control Flow Graph): temel blokları **düğüm**, geçişleri **kenar** yapan şema.
- Programın "yol haritası"dır.

```
[giriş] → [koşul] → [evet bloğu] → [çıkış]  
           ↘ [hayır bloğu] → [çıkış]
```

Kontrol akışı grafiği — şema

Kontrol akışı grafiği (CFG)

tersine mühendisliğin ilk çıkardığı iskelet



Düzleştirme bu iskeleti tek bir dağıtıcıya indirir: 'hangi blok ne zaman' bilgisi kaybolur.

Derleme bayrağı (flag)

- **Derleme bayrağı:** derleyiciye verilen seçenek (ör. `-O2` , `-DGUNLUK_ACIK`).
- Aynı kaynağı farklı bayraklarla farklı derleyebilirsiniz (ör. günlük / günlüksüz).

Günlük (log) nedir?

- **Günlük:** programın çalışırken yazdığı bilgi mesajları (`printf("...")`).
- Sorun: hassas bilgi ya da iç durum günlüğe düşerse, saldırgan okur.

Şimdi hazırız

Artık şu terimleri biliyoruz:

kaynak · derleyici · ikili · tersine derleme · `strings` · sembol · hata ayıklayıcı · XOR · entropi · dal · temel blok · CFG · günlük

Bunları bu hafta boyunca **sürekli** kullanacağız. Takıldığınızda bu bölüme dönün.

Bir cümlede bu haftanın sorusu

Programı kullanıcıya teslim ettim; **saldırgan artık ona sahip**. Kodumu okumasını ve değiştirmesini nasıl **zorlaştıırırım?**

Yanıt: **kod gizleme kuralları**. Başlıyoruz.

3. Kontrol akışı kuralları

Kontrol akışı grafiği (CFG) nedir?

Derlenmiş bir fonksiyonun **iskeleti**:

- Hangi denetim hangi sırayla?
- Hangi dal başarıya gidiyor?

Saldırganın ilk işi bu iskeleti çıkarmaktır.

Bu bölümün amacı

4. haftada düzeltirmeye **giriş** yaptık.

Bu bölümde onu **güçlendiren** kuralları ekliyoruz:

K-01 opak yüklem · K-02 aritmetik kodlama · K-03 sahte/ölü · K-04 düzeltirme (derin) · K-05 rastgele çıkış · K-06 çağrı gizleme

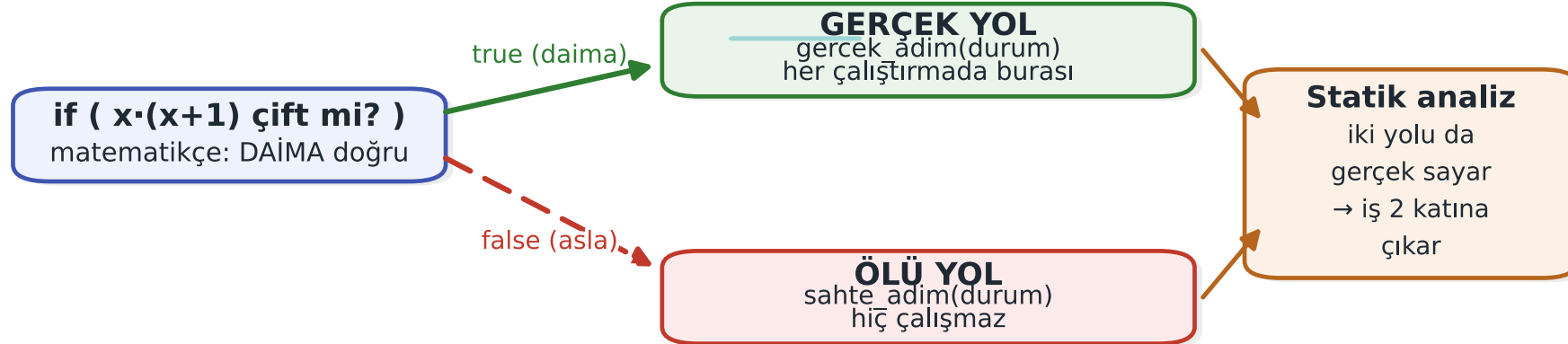
Tüm örnekler **sentetik**, tek denetim: `erisim_ver`.

K-01 · Opak yüklemeler ve döngüler

Opak yüklem — şema

Opak yüklem: gerçek yol ile ölü yol

program hep aynı çalışır, analizci iki yolu da incelemek zorunda kalır



Maliyet düşük · Sınır: klasik kalıplar araçlarca tanınır → tohumla çeşitlendirin (14. hafta).

K-01 · Neyi korur?

- Bir dalın **hangi koşulda** alındığını
- Bir döngünün **kaç kez** döndüğünü

Fikir: değeri programcının bildiği ama aracın kolay çözemediği bir ifadeyle dallan.

K-01 · Opak yüklem örneği

```
/* x*(x+1) daima çifttir → sonuç daima true */  
static int opak_dogru(unsigned x) {  
    return ((x * (x + 1)) & 1u) == 0;  
}
```

Programcı bilir: **hep true**. Statik analiz bunu kolay kanıtlayamaz.

K-01 · Nasıl kullanılır?

```
if (opak_dogru(sayac)) {  
    durum = gercek_adim(durum);    /* daima buraya girer */  
} else {  
    durum = sahte_adim(durum);    /* ölü yol: hiç çalışmaz */  
}
```

Analist iki yolu da "gerçek" sanır; zamanını sahte yola harcar.

K-01 · Opak döngü

Kılavuz döngüleri ayrı kural yapar:

- for/while **doğrudan görünmez**
- Düzleştirmenin **içine** gizlenir
- Gövde küçük bloklara bölünür
- Yineleme bir **durum değişkeniyle** yönetilir

"Şu döngü N kez dönüyor" bilgisi CFG'den okunmaz.

K-01 · Maliyet ve sınır

- **Maliyet:** düşük–orta (fazladan aritmetik/dal)
- **Sınır:** bilinen opak yüklem kalıpları otomatik tanınır
- **Karşı önlem:** çeşitlendir (bölüm 5)
- **Ölçüm:** önce/sonra temel blok sayısı

K-02 · Aritmetik kodlama

K-02 · Neyi korur?

Sabitleri ve basit hesapları.

"Bu sayı 0x1F, bu işlem bir XOR" gibi **doğrudan okunabilir** ipuçlarını gizler.

K-02 · Denk ifadeler (MBA)

Bir işlemi, sonucu değişmeyen ama karmaşık görünen denkiyle değiştir:

```
a + b    ≡    (a ^ b) + 2*(a & b)
x * 2    ≡    x << 1
```

Sonuç aynı, görünüm karmaşık. (MBA = mixed boolean-arithmetic)

K-02 · Sabiti gizle

```
/* 0x2A doğrudan yazılmaz; iki parçadan üretilir */  
static uint8_t esik(void) {  
    uint8_t a = 0x37, b = 0x1D;  
    return (uint8_t)(a ^ b);    /* = 0x2A */  
}
```

İkili dosyada `0x2A` araması sonuç vermez.

K-02 · Maliyet ve sınır

- **Maliyet:** düşük
- **Sınır:** MBA sadeleştiriciler (arybo/msynth) birçok ifadeyi açar
- **Kural:** tek başına zayıf → kontrol akışıyla **birleştir**
- **Ölçüm:** sabit için bayt/ `strings` araması boş dönmeli

K-03 · Sahte işlem ve ölü dal

Sahte işlem $\neq$ ölü dal — şema

Sahte işlem $\neq$ ölü dal

ikisi de sahte iş üretir ama farklı biçimde

SAHTE İŞLEM (bogus op)

ÇALIŞIR

sonucu kullanılmaz

analizciyi ve aracı yorar
maliyet: hafif yavaşlama

ÖLÜ DAL (dead branch)

HİÇ ÇALIŞMAZ

opak yükleme korunur

analizde gerçek görünür
maliyet: boyut

Sınav sorusu: hangisi çalışır? → Sahte işlem çalışır, ölü dal çalışmaz.

K-03 · İkisi farklıdır!

Kılavuz bunları **ayrı** kurallar yapar. Fark kritik:

- **Sahte işlem:** çalışır, sonucu değiştirmez
- **Ölü dal:** hiç çalışmaz (opak yükleme korunur)

K-03 · Karşılaştırma

	Sahte işlem	Ölü dal
Çalışır mı?	Evet	Hayır
Sonuç	Değişmez	(çalışmaz)
Amaç	Gerçeği kalabalıkta gizle	Sahte yol göster

K-03 · Sahte işlem örneği

```
crc = crc32_guncelle(crc, veri, n);  
crc ^= (sabit ^ sabit); /* = crc; sonuç aynı */
```

`sabit ^ sabit == 0` → CRC değişmez, ama kod kalabalıklaşır.

K-03 · Ölü dal örneği

```
if (opak_yanlis()) {           /* daima false */  
    sahte_kontrol();          /* hiç çalışmaz */  
}
```

Analist bu bloğu "gerçek" sanıp inceler; zaman kaybeder.

K-03 · Maliyet ve sınır

- **Maliyet:** düşük (sahte işlem) → orta (ölü dallar CFG'yi büyütür)
- **Sınır:** ölü kod eleme, opak yüklem çözülürse dalı atar
- **Bağımlılık:** opak yüklem **kalitesine** bağlı
- **Ölçüm:** ölü/gerçek dalı ayırt etme süresi

K-04 · Kontrol akışı düzleştirme

Düzleştirme: önce/sonra — şema

Kontrol akışı düzleştirme: CFG önce ve sonra

aynı program, aynı sonuç — farklı iskelet

ÖNCE: okunur akış

giriş → koşul → dallar
döngü yapısı görünür

analizci mantığı
doğrudan okur

SONRA: düzleştirilmiş

tek switch (dağıtıcı)
bloklar durum değişkeniyle
sırası gizli

yapı bilgisi KAYBOLUR

Tek başına zayıf: dağıtıcı deseni tanınır → opak yüklem + durum şifreleme ekleyin.

K-04 · Neyi korur?

Algoritmanın **yapısını**:

- Blokların doğal komşuluğunu
- "Hangi bloktan hangisine gidilir" bilgisini

K-04 · Fikir: dikey → yatay

- **Normal:** bloklar birbirini izler (dikey akış)
- **Düzleştirilmiş:** tüm bloklar tek `switch` dağıtıcıda (yatay)
- Sıra yalnız bir **durum değişkeninden** okunur

K-04 · Şablon

```
int durum = BASLA;
for (;;) switch (durum) {
    case BASLA: durum = ADIM1; break;
    case ADIM1: durum = kosul ? ADIM2 : HATA; break;
    case ADIM2: durum = BITIR; break;
    case HATA: return RED;
    case BITIR: return IZIN;
    default: return RED; /* rastgele çıkış (K-05) */
}
```

K-04 · Tek başına zayıf

4. haftada söylemiştik: bu şablon deneyimli analiste kısa sürede çözülür.

Güç, güçlendirmelerden gelir. Sırayla:

K-04 · Güçlendirme 1 — durum değerleri (K-02)

- case 1, 2, 3 yerine dağınık, aritmetikle üretilen değerler
- Ardışıklık **görünmez**
- Hangi adımın hangisini izlediği okunmaz

K-04 · Güçlendirme 2 — sahte/ölü (K-03)

- Sahte `case` 'ler eklenir
- Hangi `case` **gerçek**, ayırt etmek zorlaşır

K-04 · Güçlendirme 3 — rastgele çıkış (K-05)

- Hata, `default` dalından **öngörülemeyen** durumla çıkar
- Başarısızlığın **nerede** olduğu okunmaz

K-04 · Güçlendirme 4 — opak geçişler (K-01)

- `case` geçişleri sabit değil, **opak koşullara** bağlı
- Statik izleme daha da zorlaşır

K-04 · Maliyet ve sınır

- **Maliyet:** orta–yüksek (blok/dal çok artar, başarımlar düşer)
- **Sınır:** yalnız düzleştirme → sembolik yürütmeye açılır (bölüm 5)
- **Güç:** güçlendirmelerden
- **Ölçüm:** CFG düğüm/kenar sayısı, izleme süresi

K-05 · Rastgele çıkış

K-05 · Neyi korur?

Bir denetimin **nerede** başarısız olduğunu.

Saldırgan tipik olarak:

- "Başarısızlık dalını" bulur
- Onu "başarı dalına" çevirir

K-05 · Nasıl?

```
if (!imza_gecerli(p)) {  
    durum = kararsiz_deger(); /* switch'te tanımsız → default */  
    break;  
}
```

Doğrudan `return RED` yok; durum öngörülemez bir değere gider.

K-05 · Başarı da benzer çıksın

- Başarılı çıkış da aynı yöntemle
- Böylece başarı ve başarısızlık akışta **birbirine benzer**
- Analist "işte kabul dalı" diyemez

K-05 · Maliyet ve sınır

- **Maliyet:** düşük
- **Sınır:** tek başına değil, düzleştirmeyeyle anlamlı
- **Hedef:** tek bayt yaması denetimi atlatmaya **yetmesin**
- **Ölçüm:** "başarısızlık dalını bul ve çevir" süresi

K-06 · Çağrı ve bağımlılık gizleme

K-06 · Neyi korur?

İpuçlarını: "Şu fonksiyon `memcmp` çağırıyor → bir karşılaştırma yapıyor."

Standart kütüphane çağrıları ne yaptığınızı **söyler**.

K-06 · Nasıl?

- Kritik yolda `memcmp` yerine **kendi** sabit-zamanlı `esit_mi`
- Bağlantı çözümleyicide tanınan ad **görünmez**
- **Sahte parametreler:** hiç kullanılmayan parametreler ekle → imza yanıltıcı

K-06 · Sabit zamanlılığı bozma

Kendi karşılaştırmanı yazarken **sabit zamanlı** olsun (3. hafta).

- Erken çıkışlı karşılaştırma → **yan kanal** açar
- Koruma eklerken **yeni açık** yaratma

K-06 · Maliyet ve sınır

- **Maliyet:** düşük–orta (kendi sürümleri bakım ister)
- **Sınır:** davranış analizi ne yaptığını yine gösterebilir → bu bir **gecikme**
- **Ölçüm:** ikili dosyada tanınan kütüphane çağrısı sayısı

Bölüm 3 — özet

- **K-01** opak yüklem/döngü · **K-02** aritmetik kodlama
- **K-03** sahte işlem $\neq$ ölü dal
- **K-04** düzleştirme (güçlendirmelerle)
- **K-05** rastgele çıkış · **K-06** çağrı gizleme

Hiçbiri tek başına yeterli değil; **birlikte** ve **ölçülmüş**.

Bölüm 3 — kısa sinama

1. Sahte işlem ile ölü dal farkı?
2. Düzleştirmeyi güçlendiren 4 kural?
3. Rastgele çıkış hangi saldırıyı zorlaştırır?

Bölüm 3 — cevaplar

1. **Sahte işlem** çalışır ama sonucu değiştirmez; **ölü dal** opak yüklemle hiç çalışmaz.
2. Durum değerlerini gizle (K-02) · sahte/ölü blok (K-03) · rastgele çıkış (K-05) · opak yüklem geçişleri (K-01).
3. "Başarısızlık dalını bul ve başarıya çevir" saldırısını; artık **tek-bayt yama yetmez**.

4. Veri ve program bütünü kuralları

Kontrol akışı $\neq$ veri

- **Kontrol akışı** (bölüm 3): algoritmanın **yapısını** gizler
- **Veri gizleme** (bu bölüm): programın işlediği **değerleri** gizler

Dizgeler, sabitler, tablolar, değişkenler.

K-07 · Dize kodlama

K-07 · Neyi korur?

İkili dosyadaki **okunabilir metinleri**.

En ucuz saldırı adımı: `strings` .

`"Lisans gecersiz"` ya da bir URL → saldırgana nereye bakacağını söyler.

K-07 · Fikir

- Hassas dizge **derleme öncesi** kodlanır (ör. XOR)
- İkili dosyada **kodlanmış** durur
- **Kullanım anında** çözülür
- İş biter bitmez **silinir**

K-07 · Örnek (sentetik)

```
static const uint8_t GIZLI[] = {0x3B,0x2A,0x2E,0x2E,0x2D};  
void kullan(void) {  
    char tmp[sizeof GIZLI];  
    for (size_t i=0;i<sizeof GIZLI;i++)  
        tmp[i] = GIZLI[i] ^ 0x5A;    /* çöz */  
    isle(tmp, sizeof GIZLI);  
    memset_s_benzeri(tmp, sizeof tmp); /* hemen sil */  
}
```

K-07 · Kritik sınır

- Çalışırken çözülmüş dize **bellekte açık**
- Çözme anahtarı da ikili dosyada
- Bu bir **statik tarama** önlemidir

Sahada: çözme anahtarı **parçalanır ve dağıtılır**, çözme fonksiyonu ek denetimlerle korunur.

K-07 · Dize gizleme $\neq$ anahtar saklama

Tekrar: dize gizleme `strings` 'i durdurur, **anahtar** korumaz.

- Anahtar için $\rightarrow$ whitebox (11. hafta) ya da donanım
- **Ölçüm:** `strings` çıktısında hassas dize **bulunmamalı**

K-08 · Opak boolean

K-08 · Sorun: düz 0/1 tehlikeli

Bir denetimin sonucunu düz 0 / 1 döndürmek:

- Saldırgan **tek baytı** çevirir
- "Başarısız" → "başarılı" olur

K-08 · Fikir

Doğru/yanlış değerini:

- **Birden çok değere** bağla
- Tek bayt yamasıyla çevrilemeyecek biçimde tut

Fonksiyonlar düz boolean yerine **opak dönüş kodu** verir.

K-08 · Örnek (kavram)

```
typedef struct { uint32_t a, b; } Karar;  
static Karar izin_ver(void) {  
    return (Karar){ 0xA3C1u, 0x5C3Eu }; /* a ^ b == 0xFFFF */  
}  
static int karar_izin_mi(Karar k) {  
    return (k.a ^ k.b) == 0xFFFFu;  
}
```

Çağırın taraf **iki alanı da** doğrular.

K-08 · Maliyet ve sınır

- **Maliyet:** düşük
- **Sınır:** yeterince incelenirse çözülür
- **Amaç:** tek-bayt yamayı ve kaba dal-çevirmeyi engellemek
- **Ölçüm:** sonucu çevirmek için gereken **bağımsız nokta** > 1

K-09 · Değişken bölme ve yeniden yapılandırma

K-09 · Neyi korur?

Hassas bir değişkenin bellekteki **tanınabilir izini**.

"32 baytlık blok = AES anahtarı" gibi kalıplar yol gösterir.

K-09 · Nasıl?

- Değişkeni **böl** (32-bit'i iki 16-bit paydan üret)
- Birden çok değişkeni tek söze **birleştir**
- Dizileri yeniden düzenle
- Tampon boyut/sıra/yerleşimini değiştir
- Kullanımdan sonra **rastgele değerle doldur**

K-09 · Maliyet ve sınır

- **Maliyet:** düşük–orta
- **Sınır:** yalnız statik/kalıp analizini yavaşlatır
- **Ölçüm:** hassas tamponun kalıptan tanınma süresi

Program bütünü: K-10, K-11, K-12

K-10 · Sanallaştırma (kavram)

Neyi korur? Bir fonksiyonun **makine kodunu**.

Nasıl? Fonksiyon → özel bir **VM'in bayt koduna**; ikiliye bayt kodu + küçük yorumlayıcı gömülür.

Saldırgan tanıdık kodu değil, önce çözmesi gereken **özel komut kümesini** görür.

K-10 · Maliyet ve sınır

- **Maliyet: yüksek** (yorumlanan kod yavaş; bakım pahalı)
- **Kullanım:** yalnız en kritik, küçük, sık değişmeyen fonksiyonlar
- **Sınır:** VM bir kez çözülürse tüm korunanlar açılır; çeşitlendirilmezse tüm kopyalar aynı
- xiv. hafta: Tigress **Virtualize**

K-11 · Derleyici tabanlı (O-LLVM)

Neyi korur? Yukarıdaki kuralları, **el ile değil** derleyici geçişiyle.

- LLVM tabanlı gizleyiciler: düzleştirme, sahte akış, komut değiştirme geçişleri
- Kaynak **okunur** kalır; koruma bir derleme seçeneği
- Her sürümde farklı **tohum** → çeşitlendirme kolay

K-11 · Maliyet ve sınır

- **Maliyet:** orta (geçişlere bağlı)
- **Sınır:** bilinen geçiş kalıpları tanınır; araç sürümüyle güncel kal
- **Ölçüm:** farklı tohumların ürettiği ikililer arası fark

K-12 · Dinamik şifreleme (kavram)

Neyi korur? En hassas kod bölümlerini, ikilide **şifreli** tutarak.

Nasıl? Bölüm şifreli durur → yalnız çalışacağı an çözülür → sonra yeniden şifrelenir/silinir.

K-12 · Risk

- OS bellek korumalarıyla (**DEP/NX**, **W^X**) doğrudan çatışır
- Yanlış uygulanırsa hem çöker hem **yeni açık** yaratır
- Çalışırken bir an kod bellekte açık → döküm yakalar

K-12 · Kural

Bir koruma eklerken başka bir korumayı **kapatıyorsan**, net kazancı ölç ve ödünleşim kaydına yaz.

Uygulamada çok seçici, küçük bölümlerde kullanılır.

Java/yönetilen taraf (5. haftadan)

- Bu haftanın kuralları **native** (C/C++) için
- Yönetilen dilde karşılığı: **ProGuard/R8** ad gizleme + ölü kod eleme + küçültme
- `-keep` ve reflection ile çağrılan API'ler

İki taraf birlikte çalışır

- Java tarafı R8 ile gizlenir
- Native taraf bu haftanın kurallarıyla
- **Çapraz denetim:** native Java'nın bütünlüğünü, Java native'in sağlamasını denetler (6. hafta RASP)

Bölüm 4 — özet

- **K-07** dize kodlama · **K-08** opak boolean · **K-09** değişken bölme
- **K-10** sanallaştırma (pahalı) · **K-11** derleyici (O-LLVM) · **K-12** dinamik (riskli)

Veri gizleme, kontrol akışı gizlemeyi **tamamlar**.

Bölüm 4 — kısa sinama

1. Dize gizleme anahtarı korur mu?
2. Opak boolean neden tek-bayt yamayı zorlaştırır?
3. Sanallaştırmanın iki büyük maliyeti?

Bölüm 4 — cevaplar

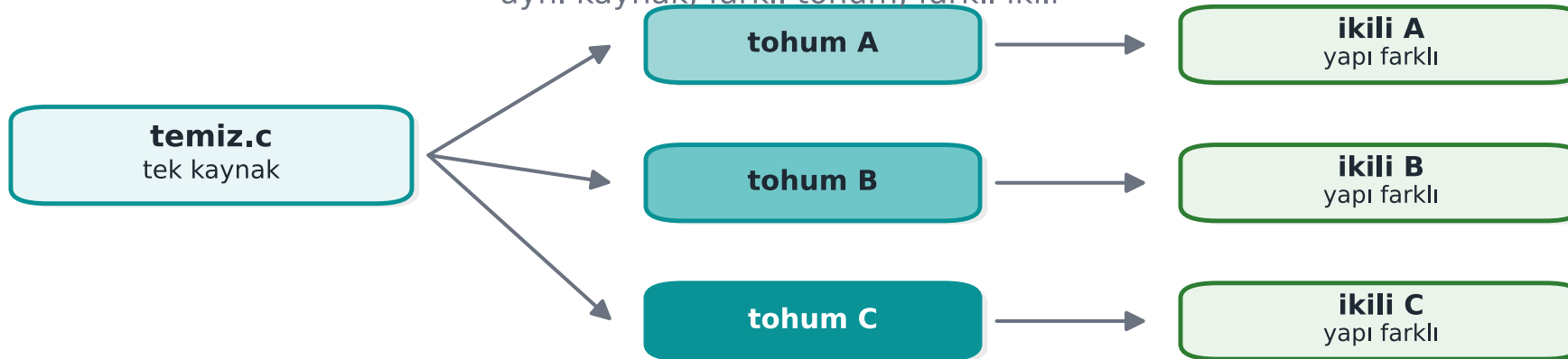
1. **Hayır**. Çözülen dize bellekte açıktır, çözme anahtarı içeridedir; yalnız statik (`strings`) taramayı durdurur.
2. Sonuç birden çok değere bağlıdır; sonucu çevirmek için **>1 bağımsız noktayı** değiştirmek gerekir.
3. Yüksek **başarım** (onlarca kat yavaş) ve yüksek **bakım** maliyeti; bu yüzden yalnız küçük/kritik fonksiyonlara.

5. Çeşitlendirme ve ölçme

Çeşitlendirme — şema

Çeşitlendirme: tek kırık her yeri açmasın

aynı kaynak, farklı tohum, farklı ikili



Davranış birebir aynı; bir kopyaya yazılan yama diğerlerinde ÇALIŞMAZ.

Çeşitlendirme: Kural 2'yi hatırla

Bir kopyada işe yarayan saldırı, bütün kopyalarda işe yaramasın.

Saldırgan bir kopyayı kırıp yamayı/betiği **herkese** dağıtabiliyorsa, bir kırık her yeri açar.

Çeşitlendirme nedir?

Aynı kaynaktan **davranışça eş, yapıca farklı** ikili dosyalar üretmek.

Opak yüklemeler, sahte bloklar, durum değerleri kopyadan kopyaya **değişir**.

İki tür çeşitlendirme

- **Uzayda:** her yapı/dağıtım farklı **tohumla**
 - Bir kopyaya yazılan saldırı diğerinde çalışmaz
- **Zamanda:** her sürüm yeni düzenle
 - Eski sürüme yazılan saldırı yeni sürümde bozulur

10. haftadaki anahtar/sürüm yenilemeyle birlikte çalışır.

Çeşitlendirme neyi engeller?

- Gizlemenin **gücünü** artırmaz (bir kopyayı kırmak yine mümkün)
- Ama **ölçeklenmesini** engeller

14. hafta: Tigress --Seed , RandomFuns ile araçla.

Çeşitlendirme etkinliği: basit ölçüt

- Aynı kaynağı **iki farklı tohumla** derle
- Korunan fonksiyonların **bayt farkını** ölç
- Fark ne kadar yüksekse, bir saldırının diğerinde çalışma olasılığı o kadar düşük

Ölçütü S9'a yaz.

Gizlemeyi ölçmek

Neden ölçmek?

Bir koruma kararını **savunmak** için.

Collberg'in çerçevesi: dört boyut.

Bu dördü 13. haftadaki **saldırı potansiyeli** puanlamasının temeli.

Ölçüt 1 — Güç (potency)

Soru: insana ne kadar anlaşılmaz?

Nasıl: karmaşıklık ölçütleri — CFG düğüm/kenar, çevrimsel karmaşıklık, iç içe geçme derinliği (önce/sonra).

Ölçüt 2 — Dayanıklılık (resilience)

Soru: otomatik araca ne kadar dayanır?

Nasıl: bir deobfuscation aracının (sembolik yürütme, sadeleştirici) korumayı **geri açma** süresi/başarısı.

Ölçüt 3 — Gizlilik (stealth)

Soru: gizleme kendini ele veriyor mu?

Nasıl: gizlenmiş kod normal koddan istatistiksel ayırt edilebiliyor mu?

Ayırt edilirse saldırgan nereye bakacağını bilir.

Ölçüt 4 — Maliyet (cost)

Soru: bize ne kadara mal oluyor?

Nasıl: boyut artışı, çalışma süresi, bellek, bakım, hata ayıklama zorluğu.

Dört ölçüt ödünleşir

Artırınca	Olan
Güç ↑ / dayanıklılık ↑	Maliyet ↑
Karmaşıklık ↑	Gizlilik ↓ (daha çok dikkat çeker)

İyi karar: **varlığın değerine** göre dört boyutu dengele.

Karar kuralı (S9'a yazılır)

Varlık değeri DÜŞÜK → hafif, ucuz (K-01, K-02, K-07)

Varlık değeri YÜKSEK → katmanlı (K-04 + K-05 + K-08 + çeşitlendirme)

Her katman: kazanç (güç/dayanıklılık) vs. maliyet (boyut/hız) → ölç ve yaz

Deobfuscation ve dayanıklılık

Neden karşı tarafın araçlarını bilmeliyiz?

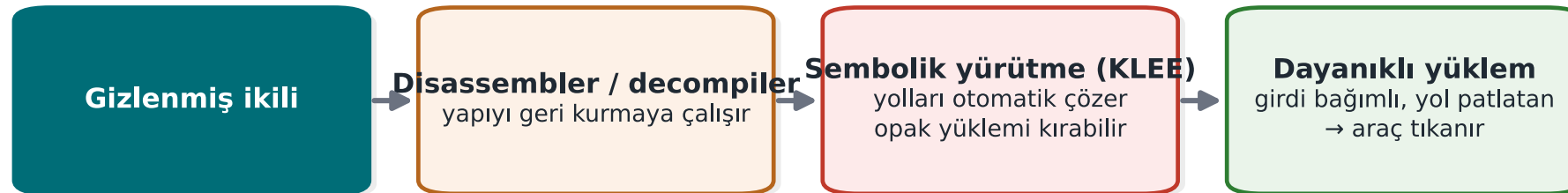
Dayanıklılığı doğru ölçmek için.

Bunları **savunmayı sinamak** için öğreniriz — saldırı için değil.

Karşı tarafın araçları — şema

Karşı tarafın araçları

dayanıklılık, bu araçlara karşı ölçülür



Klasik kalıplar kütüphanelerde tanınır → tohumla çeşitlendirin (14. hafta).

Araç 1 — sembolik yürütme (KLEE)

- Program yolları matematiksel kısıt olarak çözülür
- Opak yüklem ve düzleştirme bu yolla açılabilir

Dayanıklılık kuralı: opak yüklemeleri çözmesi pahalı (çarpanlara ayırma, karma) yapılara bağla, durum uzayını büyüt — ama **maliyeti ölç**.

Araç 2 — MBA sadeleştirici

- Aritmetik kodlamayı (K-02) açar

Kural: aritmetik kodlamayı tek başına bir sır katmanı **sayma**; kontrol akışıyla birleştir.

Araç 3 — kalıp tanıma

- Bilinen kütüphane/geçiş kalıplarını tanır

Kural: çeşitlendir; tek bir opak yüklem kalıbına ya da tek VM tasarımına bağlı kalma.

Ana çıkarım

Banescu vd. (Tigress + KLEE): hangi dönüşüm sembolik yürütmeye ne kadar dayanıyor, **ölçülür**.

Dayanıklılık bir **iddia** değil, **ölçülen** bir niceliktir.

"Güçlü" deme; "şu araca karşı şu kadar dayandı" de.

6. Katmanlı savunma, proje, kapanış

Gizlemenin katmanlı savunmadaki yeri

Katman	Hafta	İlişki
Bellek güvenliği/derleme	4	Yerine geçmez; birlikte
RASP	6	Gizleme RASP'ı gizler; RASP gizlemeyi korur
Whitebox	11	Anahtarın asıl koruması; gizleme kabuk
Anahtar yenileme	10	"Yenilenene kadar" zaman
Sertifikasyon	12–13	Geciktirme puanlanır

Sahadan bir gözlem

Sertifikalı üründe native önlemler bu haftanın kurallarının **tümünü** kapsar.

Ama belge her önlemin yanına: "**tek başına güçlü değildir.**"

Değerlendiriciyi ikna eden: birliktelik + çeşitlendirme + **ölçülmüş** geciktirme.

Proje · S9 (ileri sağlamlaştırma) — 1

1. Koruma tablosu

2–3 kritik bölüm için (lisans, anahtar türetme, bütünlük) kural şablonunu doldur:

neyi korur · tehdit · nasıl · maliyet · sınır · ölçüm

Proje · S9 — 2

2. Ölçüm

En az bir teknik için önce/sonra:

- `strings` çıktısındaki hassas dize sayısı
- CFG düğüm sayısı
- ikili boyut, bir işlemin süresi

Proje · S9 — 3, 4

3. Çeşitlendirme kararı: evet/hayır + **gerekçe**

4. Sınır ve kalan risk: her koruma neyi **korumaz**?

"Bu dize gizleme anahtarı korumaz; anahtar için S8 whitebox/donanım."

Değerlendirici gözüyle

S9'da aranan: "çok teknik kullandım" **değil**.

Aranan: **her tekniğin gerekçesi ve ölçüsü**.

Ölçülmemiş koruma = iddia. Kanıt değil.

Kendini sınaama (1–6)

1. MATE ne? Kriptografiyi neden tek başına yetersiz bırakır?
2. "Pahalı kılar" — varlık değeri örneğiyle?
3. Sahte işlem $\neq$ ölü dal?
4. Düzleştirme neden tek başına zayıf? 3 güçlendirici?
5. Rastgele çıkış hangi saldırıyı zorlaştırır?
6. Dize gizleme anahtarı korur mu?

Kendini sinama — cevaplar (1–6)

1. **MATE = Man-At-The-End**: saldırgan cihaza tam sahip (bellek, debugger, anahtar). Kripto güvenli uç varsayar; MATE'de anahtar **çalışırken** açığa çıkar → kripto tek başına yetmez.
2. Gizleme kırılmazlık değil, saldırıya **daha çok zaman/beceri/araç** gerektirir. 100 TL'lik içeriği kırmak 10.000 TL emek isterse saldırgan vazgeçer.
3. **Sahte işlem** çalışır ama sonucu kullanılmaz (analizciyi yorar); **ölü dal** opak yüklemle **hiç çalışmaz**.
4. Düzleştirmede dispatcher deseni tanınır, durum değişkeni izlenir. Güçlendirici: **opak yüklem · durum değişkeni şifreleme · sahte durum/bloklar + rastgele çıkış**.
5. **Desen/eşleştirme ve otomatik betik** saldırılarını (aynı giriş→aynı kalıp bekleyen) ve sembolik yürütmeyi zorlaştırır.
6. **Hayır**. Dize/tablo gizleme statik `strings` 'i zorlaştırır ama anahtar **çalışırken bellekte** açığa çıkar → whitebox/HSM gerekir.

Kendini sınaama (7–12)

7. Sanallaştırmanın iki maliyeti?
8. Kendini değiştiren kod hangi OS korumasıyla çatışır?
9. Dört ölçüt; hangileri ödünleşir?
10. Çeşitlendirme gücü mü ölçeklenmeyi mi engeller?
11. KLEE hangi kuralları zorlar? Dayanıklılık nasıl artar?
12. Bir bölüm için kural şablonunu doldur.

Kendini sına — cevaplar (7–12)

7. Büyük **performans** cezası (yorumlayıcı yavaş) + **boyut** artışı (VM + bytecode); bakım/hata ayıklama zorluğu.
8. **W^X / DEP-NX** (yazılabilir+çalıştırılabilir bellek yasağı); kod sayfasını yazmak `mprotect` / `VirtualProtect` ister, engellenir/şüphelidir.
9. **Güç · dayanıklılık · gizlilik · maliyet**. Güç/dayanıklılık $\uparrow \rightarrow$ maliyet $\uparrow$ ve gizlilik $\downarrow$ (anormal görünür). Başlıca ödünleşim: **dayanıklılık** $\leftrightarrow$ **maliyet**.
10. **Ölçeklenmeyi** engeller — tek kopyanın gücünü artırmaz; bir kırık tüm kopyalara **yayılmaz** (saldırının yeniden kullanımını kırar).
11. **KLEE** sembolik yürütme; opak yüklem/düzleştirmeyi çözebilir. Kural: yüklemeleri **sembolik yürütmeye dayanıklı** (girdi-bağımlı, çözülmesi zor) yap; yol patlatan yapılar dayanıklılığı artırır.
12. Örnek: **Kural** [düzleştirme] $\rightarrow$ **Amaç** [akışı gizle] $\rightarrow$ **Nasıl** [Tigress Flatten + opak yüklem] $\rightarrow$ **Ölçüm** [boyut + %X, hız - %Y, komut $N \rightarrow M$]. Ölçüm satırı şarttır.

Özet: bu haftanın tek cümlesi

Gizleme kırılmazlık vermez, **maliyet** yükseltir; gücü **birliktelikten, çeşitlendirmeden ve ölçülmüş** olmaktan gelir.

Sonraki hafta

10. hafta — Sertifikalar ve kriptografik yöntemler

"Gizleme anahtarı korumaz" dedik → anahtarların doğru seçimi, yaşam döngüsü, PKI.

11. hafta whitebox · 14. hafta Tigress (bu kuralların otomatik hali).

Ek A · Uçtan uca işlenmiş örnek

Amaç

Tek bir sentetik denetimi (`erisim_ver`) alıp **katman katman** koruyacağız.

Her adımda: ne eklendi · ne kazandık · ne ödedik.

Adım 0 · Korumasız başlangıç

```
int erisim_ver(const char *jeton) {  
    if (jeton_gecerli(jeton))  
        return IZIN;      /* tek dal, tek dönüş */  
    return RED;  
}
```

Zayıflık: tek dal, `strings`, tek bayt yamasıyla atlanır.

Adım 0 · Saldırgan ne yapar?

- `strings` → `IZIN` / `RED` ve ilgili dizgeler
- Tersine derle → tek `if`, iki dönüş
- Başarısızlık dalını başarıya çevir (tek bayt)

Süre: dakikalar. **Uzmanlık:** düşük.

Adım 1 · K-07 dize/sabit kodla

```
/* IZIN/RED ve dizgeler artık düz görünmez */  
static int izin_kodu(void){ return 0x9E ^ 0x9F; } /* =1 */
```

- **Kazanç:** strings hassas metni bulmaz
- **Maliyet:** çok düşük

Adım 1 · Ölçüm

Ölçüt	Adım 0	Adım 1
<code>strings</code> hassas dize	3	0
İkili boyut	temel	+%0.5

İlk katman en ucuz, en yüksek getiri.

Adım 2 · K-02 aritmetik kodla

```
/* karşılaştırma karmaşık ama denk bir ifadeye döner */  
int esik = ((a ^ b) + 2*(a & b)); /* = a + b */
```

- **Kazanç:** sabit/hesap doğrudan okunmaz
- **Maliyet:** düşük

Adım 3 · K-04 düzleştir

```
int durum = BASLA;
for(;;) switch(durum){
    case BASLA: durum = kontrol()?ADIM:HATA; break;
    case ADIM:  durum = BITIR; break;
    case HATA:  return RED;
    case BITIR: return IZIN;
}
```

Tek `if` yok; sıra durum değişkeninde.

Adım 3 · Ne değişti?

- CFG artık "yatay" — bloklar tek switch'te
- "Hangi dal başarı" doğrudan okunmaz
- **Maliyet:** orta (blok/dal artar, biraz yavaşlar)

Adım 4 · K-05 rastgele çıkış

```
if (!kontrol()) { durum = kararsiz_deger(); break; } /* →default */
```

- Hata **nerede** yakalandı, akıştan okunmaz
- Tek bayt yaması artık yetmez

Adım 5 · K-03 sahte + ölü

```
crc ^= (s ^ s);          /* sahte işlem: sonuç aynı */  
if (opak_yanlis()) sahte(); /* ölü dal: çalışmaz */
```

- Gerçek mantık kalabalık içinde
- **Maliyet:** orta

Adım 6 · K-08 opak boolean

```
Karar k = izin_ver();           /* a^b==0xFFFF */  
if (karar_izin_mi(k)) uygula();
```

- Sonuç düz 0/1 değil → tek bayt çeviremez
- Çevirmek için **birden çok** nokta gerekir

Katmanların birikimli etkisi

Adım	Kural	Saldırı süresi (nispi)	Maliyet
0	—	1×	—
1	K-07	2×	çok düşük
2	K-02	3×	düşük
3	K-04	8×	orta
4–6	K-05/03/08	20×+	orta

(Nispi, örnek amaçlı; gerçek değerler ölçülür.)

Adım 7 · K-10 sanallaştırma?

- Yalnız **en kritik** fonksiyon için
- **Maliyet:** yüksek (onlarca kat yavaş)
- Çoğu denetim için **gereksiz**

Kural: değeri düşükse buraya gelme.

Uçtan uca: çıkarım

- Her katman saldırıyı biraz daha **pahalı** yaptı
- Her katman bir **maliyet** ekledi
- Nerede durduğun **varlığın değerine** ve **ölçtüğüne** bağlı

Bu tablo, S9'a koyacağın koruma tablosunun ta kendisidir.

Ek B · CFG okuma ve ölçüm

CFG nedir, nasıl bakılır?

- **Düğüm:** temel blok (dalsız komut dizisi)
- **Kenar:** bloklar arası geçiş (dal)
- Tersine derleyici (ör. Ghidra) CFG'yi çizer

Gizlemenin **gücünü** düğüm/kenar sayısıyla ölçeriz.

Düzleştirme öncesi (kavram)

```
[giris] → [kontrol] → [basari] → [cikis]  
           ↘ [hata] → [cikis]
```

Az düğüm, okunur akış.

Düzleştirme sonrası (kavram)

$[giris] \rightarrow [switch] \Rightarrow \{case1, case2, \dots, caseN, sahte1..k\}$

Çok düğüm, tek merkez; "sonra hangi blok" okunmaz.

Güç ölçümü örneği

Ölçüt	Önce	Sonra
Temel blok	5	34
Kenar	6	60
Çevrimsel karmaşıklık	3	28

Rakamları tersine derleyiciden alıp S9'a yazarsınız.

Maliyet ölçümü örneği

```
size ./program_temiz ./program_gizli # boyut  
# süre: aynı girdiyle N kez çalıştır, ortalama
```

Ölçüt	Önce	Sonra
Boyut	100 KB	128 KB
İşlem süresi	1.0×	1.7×

Ölçümü yorumlamak

- Güç $\uparrow$ (34 blok) ve maliyet $\uparrow$ (%28 boyut, $1.7 \times$ süre)
- Bu **ödünleşim** beklenir
- Karar: kazanç, varlık değerine göre kabul edilebilir mi?

"Güçlü" deme; **bu sayıları** göster.

Ek C · Dayanıklılığı artırmak

Sembolik yürütmeye karşı — adım adım

1. Yalnız düzleştirme → çözücü yolları çözer
2. Opak yüklemi **çözmesi pahalı** yapıya bağla
3. Durum uzayını büyüt (daha çok case)
4. **Maliyeti ölç** — üstel dayanıklılık, üstel maliyet olabilir

Kalıp tanımaya karşı — adım adım

1. Tek opak yüklem kalıbı → tanınır
2. Birden çok kalıp kullan
3. **Çeşitlendir** (tohum)
4. Her sürümde kalıpları değiştir (zamanda)

MBA sadeleştiriciye karşı

1. Aritmetik kodlamayı **tek başına** sayma
2. Kontrol akışıyla **iç içe** kullan
3. Kodlanmış sabitleri opak yüklemelere bağla

Katmanların **birbirine bağlı** olması, ayrı ayrı çözülmesini zorlaştırır.

Dayanıklılık: altın kural

Bir sayı üretin:

"Şu dönüşüm hattı, şu araca karşı, şu boyutta bir problemi şu sürede çöz dürmedi."

İddia değil, **ölçüm**.

Ek D · Terimler ve kapanış

Terimler sözlüğü (1)

Terim	Anlam
MATE / beyaz kutu	Programa sahip saldırgan
Opak yüklem	Değeri programcının bildiği, aracın zor çözdüğü koşul
Düzleştirme	Akışı tek switch'e taşıma
Sahte işlem	Çalışan ama sonucu değiştirmeyen kod
Ölü dal	Hiç çalışmayan (opak yüklü) dal

Terimler sözlüğü (2)

Terim	Anlam
Opak boolean	Tek baytla çevrilemeyen doğru/yanlış
Sanallaştırma	Fonksiyonu özel VM bayt koduna çevirme
Çeşitlendirme	Aynı kaynaktan yapıcı farklı kopyalar
Güç/dayanıklılık/gizlilik/maliyet	Gizleme ölçütleri

Kaynaklar

- **Cookbook** (Viega & Messier) Bölüm 12 — anti-tampering
- **Güvenli programlama teknik kılavuzu** — native karşı önlemler
- Collberg & Nagra, *Surreptitious Software* — taksonomi, ölçme
- Schrittwieser vd. (ACM CSUR 2016) — genel bakış, deobfuscation
- Banescu vd. — Tigress + KLEE dayanıklılık ölçümü
- Obfuscator-LLVM — derleyici tabanlı

Bu haftadan projeye taşınanlar

- **S9:** koruma tablosu (kural şablonu) + ölçüm + çeşitlendirme kararı + kalan risk
- **S15:** (14. haftada) gizlemeyi derleme hattına koymak
- **S16:** (12. haftada) gizlemenin davranışı bozmadığının testi

Son söz

Gizleme bir kaledir değil, bir **geciktirme katmanıdır**.

Gücü: **birliktelik + çeşitlendirme + ölçüm**.

Anahtar için: **kripto** (10) ve **whitebox** (11). Otomasyon: **Tigress** (14).